DAY 16 — BACKEND DEVELOPMENT INTRODUCTION

Internship Task Answers | Python Backend • Django • Django REST Framework

Objective: Understand the role of backend development, learn how a frontend communicates with a server, and create a basic Django project that can later provide APIs to a React application.

1. What is Backend Development?

Backend development is the server-side part of a web application. It handles the work that users normally do not see directly, such as receiving requests, applying business logic, validating data, communicating with a database, managing authentication, and sending responses to the frontend.

Example: When a user logs in, the React frontend sends the entered email and password to the backend. The backend checks the details, communicates with the database, and returns a success or error response.

Main responsibilities: API development, database operations, authentication and authorization, validation, security, and application logic.

2. Client–Server Communication

A client is usually a web browser or frontend application. A server is a computer or program that receives requests and provides responses. In a web application, the frontend and backend communicate over HTTP or HTTPS.

```
React Frontend (Client)
        |
        | HTTP request (URL, method, headers, data)
        v
Django Backend (Server)
        |
        | Business logic / database query
        v
Database (e.g., MySQL or SQLite)
        |
        v
Django sends an HTTP response (often JSON)
        |
        v
React displays the result to the user
```

Communication flow: User action → frontend sends request → backend processes request → backend may read/write database → backend sends response → frontend updates the screen.

3. HTTP Methods

HTTP methods tell the server what action the client wants to perform.

Method	Purpose	Example
GET	Retrieve data	Get a list of students
POST	Create new data	Register a student
PUT	Replace/update a complete resource	Update all student details
PATCH	Partially update a resource	Change only the email

Method	Purpose	Example
DELETE	Delete a resource	Delete a student record

4. HTTP Status Codes

An HTTP status code tells the client the result of its request.

• **200 OK:** Request completed successfully.

• **201 Created:** A new resource was created successfully.

• **204 No Content:** Request succeeded and there is no response body.

• **400 Bad Request:** Request contains invalid data.

• **401 Unauthorized:** Authentication is required or has failed.

• **403 Forbidden:** User is authenticated but does not have permission.

• **404 Not Found:** Requested resource or URL does not exist.

• **500 Internal Server Error:** An unexpected server-side error occurred.

5. REST API Concepts

REST (Representational State Transfer) is an architectural style used to design web APIs. A REST API exposes resources through URLs and commonly uses HTTP methods to perform operations on those resources. REST APIs are often stateless: each request contains the information needed to process it.

Example resource: students

```
GET     /api/students/       -> get all students
GET     /api/students/1/     -> get student with ID 1
POST    /api/students/       -> create a student
PUT     /api/students/1/     -> replace student 1
PATCH   /api/students/1/     -> partially update student 1
DELETE  /api/students/1/     -> delete student 1
```

6. Request and Response

A request is the message sent by the client to the server. It can contain a URL, HTTP method, headers, and an optional body. A response is the message returned by the server. It usually contains a status code, headers, and sometimes a response body.

Example request

```
POST /api/students/
Content-Type: application/json

{
  "name": "Aarav",
  "email": "aarav@example.com"
}
```

Example response

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 1,
  "name": "Aarav",
```

```
  "email": "aarav@example.com"
}
```

7. JSON

JSON (JavaScript Object Notation) is a lightweight text format used to exchange data between applications. It is easy for people to read and is supported by most programming languages. JSON uses key-value pairs and arrays. Keys and string values are written in double quotes.

```
{
  "id": 1,
  "name": "Aarav",
  "active": true,
  "skills": ["Python", "React"],
  "address": {
    "city": "Pune"
  }
}
```

JSON commonly represents API request and response data. Django REST Framework can convert Python data into JSON and validate incoming JSON data.

8. Introduction to Python Backend Development

Python can be used to build server-side applications and APIs. A Python backend receives HTTP requests, runs application logic, interacts with a database, and returns responses. Popular Python web frameworks include Django, Django REST Framework (built on Django), and FastAPI.

Typical architecture: React frontend → HTTP/JSON API → Python backend → database. The frontend is responsible for the user interface, while the backend is responsible for data processing, rules, and secure access to data.

9. Introduction to Django

Django is a high-level Python web framework used to build web applications quickly. It includes features such as URL routing, views, models, database integration through an ORM, an admin interface, and security-related tools.

Project: The overall Django application configuration and collection of apps.
App: A smaller module that handles one area of functionality, such as users, students, or payments.

10. Introduction to Django REST Framework (DRF)

Django REST Framework is a toolkit built on Django for creating Web APIs. It provides serializers, API views, request and response classes, authentication support, permissions, and a browsable API.

Serializer: Converts model instances or Python data into formats such as JSON, and validates incoming data.
API View: Handles an API request and returns an API response. It can implement GET, POST, PUT, PATCH, and DELETE operations.

11. Django Building Blocks

Models: Define the structure of application data. Django uses models to work with database tables through its Object-Relational Mapper (ORM).

Views: Contain the code that receives a request, performs required processing, and returns a response.

URL routing: Connects a URL pattern to the view that should handle the request.

Serializers: In DRF, convert complex data such as model objects to JSON-compatible data and validate incoming data.

API Views: DRF views handle API requests and return API responses, often using serializers and models.

12. Virtual Environments and pip

A virtual environment is an isolated Python environment for a project. It keeps that project's installed packages separate from other Python projects. pip is Python's package installer and is used to install libraries such as Django and Django REST Framework.

Create and activate a virtual environment (Windows PowerShell)

```
# Create a project folder and enter it
mkdir Day16_Django_Project
cd Day16_Django_Project

# Create a virtual environment
py -m venv venv

# Activate it in PowerShell
.\venv\Scripts\Activate.ps1

# Upgrade pip
python -m pip install --upgrade pip

# Install Django and Django REST Framework
pip install django djangorestframework

# Check installed packages
pip show django
pip show djangorestframework
```

If PowerShell blocks activation because of the execution policy, use the following command for the current terminal session, then activate again:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
```

To leave the virtual environment, run **deactivate**.

13. Create a Django Project

```
# Create a Django project named config
django-admin startproject config .

# Start the development server
python manage.py runserver
```

Open **http://127.0.0.1:8000/** in a browser. The Django welcome page confirms that the development server is running. Stop the server with Ctrl + C.

Important project files

• **manage.py:** Command-line utility used to manage the Django project.

• **config/settings.py:** Project settings, including installed apps and database configuration.

• **config/urls.py:** Main URL configuration.

• **config/asgi.py / config/wsgi.py:** Entry points used by ASGI and WSGI application servers.

14. Create a Django App and Connect URLs

```
# Create an app named students
python manage.py startapp students
```

Add **'rest_framework'** and **'students'** to INSTALLED_APPS in config/settings.py:

```
INSTALLED_APPS = [
    # Django default apps...
    'rest_framework',
    'students',
]
```

Create a simple view in students/views.py:

```
from django.http import JsonResponse

def home(request):
    return JsonResponse({
        "message": "Welcome to the Django backend!"
    })
```

Create students/urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Connect the app URL in config/urls.py:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('students.urls')),
]
```

Run the server and open **http://127.0.0.1:8000/api/**. Expected JSON response:

```
{
  "message": "Welcome to the Django backend!"
}
```

15. Create a Basic Model

A model defines the data that Django stores in the database. Add the following to students/models.py:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)

    def __str__(self):
        return self.name
```

Create and apply database migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Django's default database is SQLite, which is suitable for this introductory exercise. A project can be configured to use MySQL when required.

16. Create a Serializer and API View (DRF)

Create students/serializers.py:

```
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = ['id', 'name', 'email']
```

Add a basic API view to students/views.py:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .models import Student
from .serializers import StudentSerializer

@api_view(['GET', 'POST'])
def student_list(request):
    if request.method == 'GET':
        students = Student.objects.all()
        serializer = StudentSerializer(students, many=True)
        return Response(serializer.data)

    serializer = StudentSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Update students/urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('students/', views.student_list, name='student-list'),
]
```

Start the server and visit **http://127.0.0.1:8000/api/students/**. A GET request returns the student list. A POST request with JSON data creates a student. You can test both requests using the DRF browsable API page or an API client such as Postman.

17. How React Communicates with a Backend API

React sends HTTP requests to Django API endpoints. The backend processes the request and returns JSON. React reads the response and updates the user interface.

```
// Example React code using fetch()
fetch('http://127.0.0.1:8000/api/students/')
  .then(response => {
    if (!response.ok) {
      throw new Error('Request failed');
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

For a POST request, React sends JSON in the request body. In a real project, the Django backend must be configured with suitable CORS settings if the frontend and backend run on different origins. Never place database passwords or secret keys in frontend code.

18. Expected Output

• Python virtual environment created and activated.

• Django and Django REST Framework installed successfully.

• A Django project named **config** created.

• A Django app named **students** created and connected through URL routing.

• Development server runs at **http://127.0.0.1:8000/**.

• A sample API endpoint returns JSON at **http://127.0.0.1:8000/api/**.

• Basic understanding of models, views, serializers, API views, HTTP methods, and React-to-backend communication.

19. Conclusion

In this task, I learned the fundamentals of backend development and client-server communication. I understood HTTP methods, status codes, REST APIs, requests, responses, and JSON. I also learned the purpose of Django and Django REST Framework, created a virtual environment, installed the required packages, and created a basic Django project with an app and API endpoint. This forms the foundation for connecting a React frontend with a Python backend.

Note: The commands and code in this document provide a beginner-level learning example. Run them in order from the project folder, keeping the virtual environment activated.